Documentación del Proyecto Compilador
Etapa 3: Analizador Semántico (Chequeo de Tipos, Ámbitos e Integración)

1. División del Trabajo

El desarrollo de la tercera etapa del compilador, centrada en la implementación completa del Analizador Semántico y su integración con el Árbol de Sintaxis Abstracta (AST), la Tabla de Símbolos (TS) y el gestor de compilación, se realizó de manera colaborativa distribuyendo las responsabilidades de la siguiente forma:

- **Diseño e implementación del Analizador Semántico (`semantic-analizer.c/.h`):** Gribaudo Re Francisco y Francisco Andreani.
- **Integración con el Parser, AST y Tabla de Símbolos (`parser.y`, `syntax-tree.c/.h`):** Valentín Pastre Thuer, Gribaudo Re Francisco y Francisco Andreani.
- **Manejo del compilador, constantes y CLI (`compiler-manager.c/.h`, `cli.c`, `constants.h`):** Valentín Pastre Thuer.
- **Elaboración de pruebas unitarias (`test_semantic-analizer.c`, `test_syntax-tree.c`):** Valentín Pastre Thuer y Gribaudo Re Francisco.

2. Decisiones de Diseño, Suposiciones y Aclaraciones

Habiendo estructurado en la etapa anterior la recolección de datos en la Tabla de Símbolos y la construcción base del AST, el objetivo principal de esta etapa consistió en dotar al compilador de validación semántica completa y unificar el flujo de ejecución entre todos los módulos[cite: 13].

- **Desacoplamiento entre Análisis Sintáctico y Semántico:** Se mantuvo la premisa de modularización estricta. El archivo `parser.y` se encarga exclusivamente de estructurar el árbol sintáctico (`syntaxTree`) mediante los constructores de `syntax-tree.c` (`makeNode`, `makeLeaf` y `makeSymbol`), delegando la verificación de reglas contextuales, alcances y tipos al módulo `semantic-analizer.c` a través de las funciones `analyzeAst`, `analyzeExpression` y `evaluateFunctionInvoke`[cite: 13, 15].
- **Refactorización Léxico-Sintáctica:** Para agilizar la construcción de hojas con literales e identificadores en el AST, se trasladó el reconocimiento de las expresiones regulares de identificadores (`ID`) y literales (`INT`, `FLOAT`, `BOOL`) directamente a `scanner.lex`, simplificando las producciones de `parser.y`[cite: 12].
- **Centralización de Constantes (`constants.h`):** Se unificaron las enumeraciones de tipos de datos (`tInt`, `tFloat`, `tBoolean`, `tVoid`, `blank`), tipos de símbolos y etapas del compilador en `header/constants.h` para mantener consistencia entre todos los componentes[cite: 13].
- **Promoción Implícita y Advertencias de Precisión:** Se decidió soportar coerción/promoción implícita de `tInt` a `tFloat` tanto en operaciones aritméticas mixtas como en asignaciones, comparaciones de igualdad y pasaje de argumentos a funciones. En el sentido inverso (asignación de un `tFloat` a una variable `tInt`), el analizador permite la operación pero emite una advertencia explícita (*Semantic Warning: Possible loss of precision when assigning float to int*)[cite: 1].

3. Diseño y Decisiones Clave: El Analizador Semántico

El núcleo de esta entrega reside en `src/semantic-analizer.c` y `header/semantic-analizer.h`, donde se implementaron los algoritmos de recorrido y verificación sobre el AST apoyándose en la Tabla de Símbolos multinivel[cite: 13]:

- **Manejo de Ámbitos (*Scopes*) y Clasificación de Variables:** Aprovechando la estructura de lista doblemente enlazada de `symbols-tab.c`, el analizador gestiona la apertura y cierre de niveles léxicos mediante `openLevel` y `closeLevel` al ingresar o salir de funciones y bloques (`block`). En los nodos `varDeclaration`, se utiliza `findSymbol` para evitar redeclaraciones en el ámbito actual y se inspeccionan los punteros de nivel para clasificar automáticamente cada variable como `global_var` o `local_var` antes de insertarla con `addSymbolToTab`[cite: 13, 15]. Para resolver referencias a variables o funciones en expresiones y asignaciones, se utiliza `findSymbolMultiLevel`, buscando recursivamente desde el ámbito más interno hacia el global[cite: 13].
- **Validación de Funciones y Reglas sobre `main`:**
  - Al finalizar el recorrido del nodo raíz (`program`), el analizador consulta la Tabla de Símbolos para garantizar que exista obligatoriamente una función llamada `main` y que no posea parámetros formales (`paramsNode == NULL`)[cite: 13].
  - Al procesar un nodo `function`, se verifica que no esté redefinido, se enlaza su subárbol de parámetros formales en `symbol->paramsNode`, se registra la función en el nivel actual y luego se abre un nuevo nivel (`openLevel`) donde se insertan los nodos `formalParameter` verificando que no existan parámetros duplicados[cite: 13].
- **Inferencia y Chequeo de Expresiones (`analyzeExpression`):** Se implementó una función recursiva que evalúa subárboles de expresiones y retorna el `enum types` resultante[cite: 13]:
  - *Operadores Aritméticos (`addOp`, `substractOp`, `multOp`, `divOp`):* Exigen operandos numéricos (`tInt` o `tFloat`), promoviendo el resultado a `tFloat` si alguno de los operandos es flotante. El nodo `substractOp` contempla tanto la resta binaria como el menos unario[cite: 13, 14].
  - *Operador Módulo (`modOp`):* Restringido estrictamente a operandos de tipo `tInt`[cite: 13].
  - *Operadores Lógicos (`andOp`, `orOp`) y Negación (`negationOp`):* Requieren exclusivamente operandos de tipo `tBoolean`[cite: 13, 15].
  - *Operadores Relacionales (`lessOp`, `greaterOp`) y de Igualdad (`equalsOp`):* Los relacionales exigen operandos numéricos y retornan `tBoolean`. El operador de igualdad prohíbe comparaciones con `tVoid` y exige tipos iguales o numéricos compatibles entre sí[cite: 13].
- **Verificación de Sentencias de Control, Retorno e Invocaciones:**
  - *Condicionales y Ciclos (`ifNode`, `whileNode`):* Se comprueba que la expresión de condición evalúe obligatoriamente a `tBoolean`[cite: 13].
  - *Control de `returnNode`:* Mediante la variable estática `currentMethodReturnType`, se verifica que un método `tVoid` no retorne expresiones y que un método no-`void` retorne obligatoriamente un valor cuyo tipo coincida con la firma declarada[cite: 13].
  - *Invocación de Funciones (`evaluateFunctionInvoke`):* Recibe una bandera `isExpression` que permite distinguir si la función fue llamada como sentencia independiente o dentro de una expresión (prohibiendo que métodos `tVoid` se utilicen como operandos)[cite: 13]. Además, recorre en paralelo la lista de argumentos actuales (`actualParameterList`) y la lista de parámetros formales (`func->paramsNode`) validando la cantidad de argumentos y la compatibilidad de tipos posición por posición[cite: 13].

4. Integración con el Resto del Compilador

Además del analizador semántico, se realizaron modificaciones estructurales en los módulos existentes para consolidar el flujo de compilación[cite: 13]:

- **Evolución del AST (`syntax-tree.c/.h`) y `parser.y`:** Se enriqueció la estructura `Symbol` con el puntero `paramsNode` y el soporte de valores tipados[cite: 13]. En `parser.y` se utiliza la variable auxiliar `declType` en una acción intermedia para propagar el tipo de dato hacia las listas de declaraciones separadas por comas (`id_prime`)[cite: 2].
- **Gestor de Compilación (`compiler-manager.c/.h`) y CLI (`cli.c/.h`):** Se incorporó la etapa `semantic` tanto en el procesamiento de argumentos (`-target semantic`, establecida además como etapa objetivo por defecto en `returnTargetStage`) como en `executeStage` y `makeOutputFile`[cite: 8, 9, 10]. El compilador ahora genera archivos de salida diferenciados según la etapa solicitada y redirige los reportes semánticos hacia `semanticOutputFile`[cite: 6, 8].

5. Instrucciones de Compilación y Ejecución de Pruebas

El proyecto mantiene la integración con **Ceedling** (Unity + CMock) configurada en `project.yml`, generando el ejecutable final bajo el nombre `c-tds`[cite: 1].

5.1. Ejecución de Tests Unitarios con Ceedling

En esta etapa se incorporaron suites de pruebas específicas para validar tanto la construcción del AST como las reglas del analizador semántico[cite: 13]:

- **Correr tests del Analizador Semántico:**

  ```
  ./ceedling test:semantic-analizer
  ```

- **Correr tests del Árbol Sintáctico (AST):**

  ```
  ./ceedling test:syntax-tree
  ```

- **Correr la batería completa de tests del proyecto:**

  ```
  ./ceedling test:all
  ```

  (O bien utilizando el script automatizado `./run-tests.sh` que regenera Flex y Bison antes de invocar a Ceedling).

5.2. Construcción y Uso del Compilador

Para generar la versión de lanzamiento (*Release*) y ejecutar el compilador hasta la etapa semántica sobre un archivo fuente `.ctds`[cite: 8]:

```
./compile.sh
./build/artifacts/release/c-tds -target semantic prueba.ctds
```

6. Problemas Conocidos y Testing

Durante el desarrollo y la integración de esta tercera etapa se identificaron los siguientes desafíos y detalles de implementación[cite: 13]:

- **Sincronización de Ámbitos y Parámetros Formales:** Uno de los principales retos fue asegurar que la apertura y cierre de niveles en la Tabla de Símbolos (`openLevel` y

`closeLevel`) registrara el identificador de la función en el nivel global pero alojara sus parámetros formales dentro del nuevo ámbito local del método, permitiendo además detectar parámetros duplicados antes de evaluar el bloque del cuerpo[cite: 4, 6, 13].

- **Alineación de Nodos en Listas de Declaraciones:** Al construir el AST en `parser.y` para declaraciones múltiples (`var_decl` e `id_prime`), fue necesario asegurar que la estructura de nodos generada por Bison coincidiera con el recorrido esperado por el caso `varDeclaration` en `analyzeAst`[cite: 2, 6].
- **Integración Final en `semanticStage`:** En `compiler-manager.c`, la invocación a `analyzeAst(syntaxTree, ...)` dentro de `semanticStage()` se dejó preparada junto con la asignación de `semanticOutputFile = outputFilePtr`, permitiendo ejecutar las pruebas unitarias de `test_semantic-analizer.c` de forma desacoplada pasando instancias controladas del AST y de la Tabla de Símbolos[cite: 6, 8].